DSC613 Big Data Analytics - Hive Assignment

Large-Scale Data Analysis Using Apache Hive and Visualization

Name Gayathri Suresh
Roll No. 2026EMAI10031
Submission `123456_hive_assignment.zip`

All analysis (loading, cleaning, the 20 Part C questions and the Part D exports) is done in HiveQL in `hql/123456_assignment.hql`. Python is used only in Part D, on files that Hive exported.

---

1. Environment

Component	Version / setting
Operating system	Ubuntu 24.04.5 LTS
Java	OpenJDK 1.8.0_504
Hive	Apache Hive 2.3.9 (`SELECT version()` → `2.3.9 r92dd0159f440ca7863be3232f3a683a510a62b9d`)
Execution engine	MapReduce (`hive.execution.engine=mr`), local job runner (`mapreduce.framework.name=local`)
Hadoop	3.3.4 libraries in local (standalone) mode — no HDFS or YARN daemons
Metastore	Embedded Apache Derby
Warehouse	`file:/home/gayathri/hive/warehouse` (local file system)
Client	Hive CLI (`hive -f`)
How Hive was installed	Hive 2.3.9 and Hadoop 3.3.4 libraries on OpenJDK 8, run in local mode with the `hive` launcher in `~/hive/bin`. All non-default settings are in `env/hive-site.xml` (copied into Hive's `conf/` folder).
Visualisation	Python 3.13, pandas 3.0.5, matplotlib 3.11.2, Jupyter `nbconvert`

2. Folder layout

```
123456_hive_assignment/
├── README.md                          this file
├── data/
│   └── ecommerce_transactions.csv     the dataset as supplied (15,000 rows + header)
├── hql/
│   └── 123456_assignment.hql          every query for Parts A–D and the Big Data section, in order,
│                                      with question numbers as comments
├── scripts/
│   └── capture_outputs.sh             runs each numbered query on its own and saves its output in outputs/
├── env/
│   └── hive-site.xml                  Hive/Hadoop settings used for the run (local mode, Derby, MapReduce)
├── outputs/
│   ├── full_run_log.txt               console log of one run of the whole script, top to bottom
│   ├── A1_*.txt … A7*_*.txt           Part A outputs (setup, load, validation)
│   ├── A5_csv_line_count.txt          `wc -l` of the CSV (row-count cross-check)
│   ├── B1_*.txt … B9*_*.txt           Part B outputs (exploration, cleaning, validation)
│   ├── C1_total_revenue.csv …         Part C outputs, one file per question (C1–C20, plus C2a/b, C5b,
│   │   C20c_festive_quarter_share.csv   C6b, C16b, C17b/c, C20a/b/c), header row included
│   ├── D_V1_export.txt … D_V6_export.txt   Hive log of each Part D export
│   ├── BD1_*.txt … BD4_*.txt          Big Data section: EXPLAIN, table statistics, partitions
│   └── exports/
│       ├── V1_monthly_revenue/000000_0            chart inputs written by Hive with
│       ├── V2_category_revenue/000000_0           INSERT OVERWRITE LOCAL DIRECTORY
│       ├── V3_city_revenue/000000_0               (comma-separated, no header row)
│       ├── V4_payment_methods/000000_0
│       ├── V5_order_status/000000_0
│       └── V6_category_returns_vs_rating/000000_0
├── viz/
│   ├── 123456_visualisations.ipynb    Part D notebook (executed, outputs included)
│   └── charts/V1 … V6 *.png           the six charts used in the report
└── report/
    └── 123456_report.pdf              final analytical report
```

Output file names start with the question number (for example `C5_monthly_revenue.csv`). `*.csv` files hold result rows (Hive's tab separators replaced by commas); `*.txt` files hold Hive's console output exactly as printed (DESCRIBE, EXPLAIN, load and export logs).

3. How to reproduce the analysis

1. **Start Hive.** Copy `env/hive-site.xml` into your Hive `conf/` folder (or use your own configuration) and check that the CLI starts: `hive -e "SELECT version();"`. With the embedded Derby metastore nothing else needs to be started; the metastore folder is created on first use.
2. **Place the dataset.** Unzip the project into your home folder, so the CSV is at `~/123456_hive_assignment/data/ecommerce_transactions.csv`. Do not open or re-save it in Excel.
3. **Set the two paths** at the top of `hql/123456_assignment.hql` (see section 4).
4. **Run the whole script** from the project folder:

   ```
   cd ~/123456_hive_assignment
   hive -f hql/123456_assignment.hql > outputs/full_run_log.txt 2>&1
   ```

   It creates the database and tables, loads the data, cleans it, runs every query and writes the six Part D exports to `outputs/exports/`. It took about 3 minutes in local mode. Re-running it is safe: tables are dropped or overwritten before they are rebuilt.
5. **Save one output file per query** (optional; the files are already included):

   ```
   bash scripts/capture_outputs.sh
   ```

   The script cuts the `.hql` file at its `-- @output <file>` markers and runs each piece with the Hive CLI. It takes 10–15 minutes because the CLI starts once per query.
6. **Exports.** Part D uses **Method 2** of the brief: `INSERT OVERWRITE LOCAL DIRECTORY ... ROW FORMAT DELIMITED FIELDS TERMINATED BY ','`. Hive writes **no header row** with this method, so the notebook adds the column names itself (they are also listed in a comment above each export query).
7. **Charts.** Run the notebook from the `viz/` folder:

```
cd viz
jupyter nbconvert --to notebook --execute --inplace 123456_visualisations.ipynb
```

or open it in Jupyter and choose *Run All*. The PNGs are written to `viz/charts/`.

4. Paths and settings a marker may need to change

Where	Setting	Value used
`hql/123456_assignment.hql`, line 21	`SET hivevar:csv_path=...`	`/home/gayathri/123456_hive_assignment/data/ecommerce_transactions.csv`
`hql/123456_assignment.hql`, line 22	`SET hivevar:export_dir=...`	`/home/gayathri/123456_hive_assignment/outputs/exports`
`env/hive-site.xml`	metastore, warehouse and scratch folders	under `/home/gayathri/hive/`

Nothing else in the script contains a path. The notebook uses relative paths (`../outputs/...`).

Session settings set by the script itself: `hive.cli.print.header=true`, `hive.resultset.use.unique.column.names=false`, `hive.strict.checks.cartesian.product=false` (C13, C18 and C19 join a one-row subquery of averages), and, in the Big Data section only, `hive.exec.dynamic.partition=true` and `hive.exec.dynamic.partition.mode=nonstrict`.

5. Part C feature checklist

Q	Question	Required features	Used
C1	Total revenue	SUM, WHERE	✓
C2	Transactions by status	COUNT, GROUP BY	✓ (+ window `SUM() OVER ()` for %)
C3	Top categories by revenue	SUM, GROUP BY, ORDER BY, LIMIT	✓ (+ window share)
C4	Top products by revenue	SUM, GROUP BY, ORDER BY	✓
C5	Monthly revenue trend	Date functions, SUM, GROUP BY, ORDER BY	✓ `DATE_FORMAT`, `YEAR`; C5b adds `LAG`
C6	Revenue by city	SUM, GROUP BY, ORDER BY	✓
C7	Average order value by city	AVG, GROUP BY	✓ (+ `RANK() OVER`)
C8	Payment method frequency	COUNT, GROUP BY, ORDER BY	✓
C9	Cancelled / returned %	CASE WHEN, COUNT	✓
C10	Return rate by category	CASE WHEN, GROUP BY, HAVING, ORDER BY	✓ `HAVING COUNT(*) >= 500`
C11	Rating by category	AVG, MIN, MAX, COUNT, GROUP BY	✓
C12	Delivery time by city	AVG, GROUP BY	✓
C13	Cities slower than average	AVG, HAVING, subquery	✓
C14	Revenue by gender	SUM, COUNT, GROUP BY	✓
C15	Behaviour by age band	CASE WHEN, COUNT, SUM, AVG, GROUP BY	✓
C16	Top 10 customers	SUM, GROUP BY, ORDER BY, LIMIT	✓ (C16b adds `ROW_NUMBER`)
C17	Rating vs sales	JOIN, subqueries, AVG, SUM	✓ (+ `CORR` in C17b, C17c)
C18	High volume, low revenue	Subquery, SUM, HAVING	✓
C19	High revenue, low volume	Subquery, SUM, HAVING	✓
C20	Highest-revenue month	Date functions, ORDER BY, LIMIT or subquery	✓ (`LIMIT 1`; `ROW_NUMBER` per year in C20b)

6. Assumptions and known limitations

- **Revenue** is Completed orders only; **gross order value** (all statuses) is used only in B6. Average order value = revenue ÷ Completed orders.
- **Delivered order** means Completed or Returned. Delivery-time averages (C12, C13) use delivered orders only, and rating averages skip NULL ratings, which are never treated as 0.
- **No row is dropped** in cleaning. Invalid ages, ratings and delivery times become NULL; blank gender, city and payment method become `Unknown`. City rankings (C6, C7, C12, C13) list the 14 real cities; the `Unknown` city group is reported separately in C6b.
- **Age bands**: 18–24, 25–34, 35–44, 45–54, 55–64, 65–100, plus Unknown.
- **C10 threshold**: at least 500 orders per category (95% margin of error about ±2.4 points at an 8% return rate). All 8 categories pass it on this sample.
- **C18/C19 cut-offs**: the average units sold per product and the average revenue per product over the 44 products (Completed orders), computed in the query itself and printed in its output.
- **Local mode**: the warehouse is on the local file system, not HDFS, and all tasks run in one JVM. The queries and results are the same on a cluster; only the storage location and parallelism change.
- **Tested on Hive 2.3.9 only.** The script uses standard HiveQL (no version-specific functions), and subqueries are written as joined derived tables, which Hive 2.x supports without the cost-based optimiser. To run it on the brief's `apache/hive:4.0.0` Docker image, copy the CSV into the container, set

`csv_path=/tmp/ecommerce_transactions.csv` and `export_dir=/tmp/exports`, run `beeline -u jdbc:hive2://localhost:10000/ -f 123456_assignment.hql`, then `docker cp` the exports out.

- The dataset is synthetic, so business reasons given for patterns (for example the festive season in C20) are suggestions, not proven causes.

7. Sources and tools

- Apache Hive Language Manual (DDL, DML, SELECT, windowing and analytics functions, EXPLAIN).
- Apache Hadoop 3.3.4 documentation (local mode); Apache Derby.
- Python, pandas and matplotlib documentation (Part D).
- **Generative AI:** Claude (Anthropic) was used to help debug HiveQL errors and to check the wording of the report. All queries are in the script, were run in Hive, and every number in the report is taken from the files in `outputs/`.